

COMP2017 26S1 Passway to Pass

Workshop Part 2

13:00-15:00, 16 July 2026

Hao Ren

Contents

2 / 77

1 Process Memory and Lifecycle	4
2 IPC and Process Workers	21
3 Threads and Worker-Combiner	42
4 Race Conditions and Mutexes	53
5 Deadlocks	59
6 Condition Variables and Semaphores	67
7 Recursion and Exam Transfer	73

These slides may be **updated dynamically** over the coming days, so please always check this page for the latest version.

Additionally, please note that our workshop does not cover all of the semester's content. For more detailed explanations, you can visit <https://github.com/ninn-kou/COMP2017-Tutorials>.

Updates

16 July 2026: Version 1.0 - The materials are uploaded.

18 July 2026: Version 2.0 - Two practice exercises are available via the P2P page on Ed Lessons.

18 July 2026: Version 2.1 - Some formatting issues have been corrected.

18 July 2026: Version 2.2 - Attach a code snippet in Processes part.

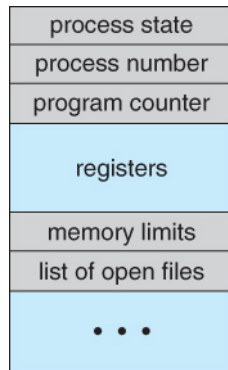
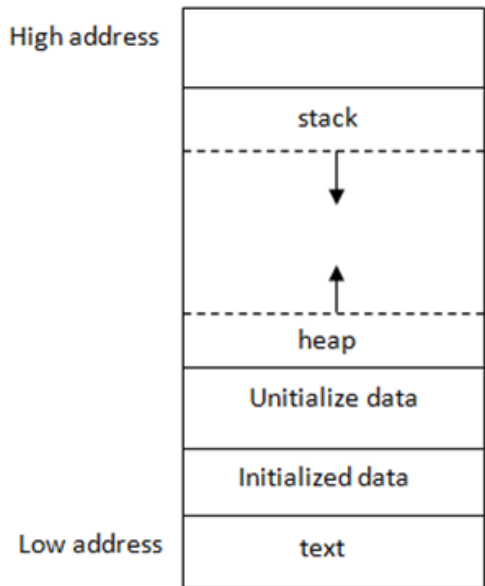
18 July 2026: Version 2.3 - Add more illustrations about pipelines.

Slides Version: 2.3

1 Process Memory and Lifecycle

Process and Its Memory Layout

5 / 77



A process is the running instance of a program. A process has its own virtual address space and its own process control block (PCB).

Process and Its Memory Layout (ii)

6 / 77

```
#include <stdio.h>
#include <stdlib.h>

int global_init = 123;           // usually in .data
int global_bss;                 // usually in .bss
static int static_init = 7;     // usually in .data
static int static_bss;          // usually in .bss

int main(int argc, char *argv[]) {
    int local = 5;               // usually on the stack
    int *heap = malloc(sizeof *heap); // points into the heap

    free(heap);
    return 0;
}
```

Process Memory and Kernel Descriptor State

7 / 77

Virtual Address Space

stack: frames and local variables

heap: dynamic allocations

data / BSS: globals and statics

code / text: instructions

Kernel-Managed Process State

PID and lifecycle state

file descriptor table

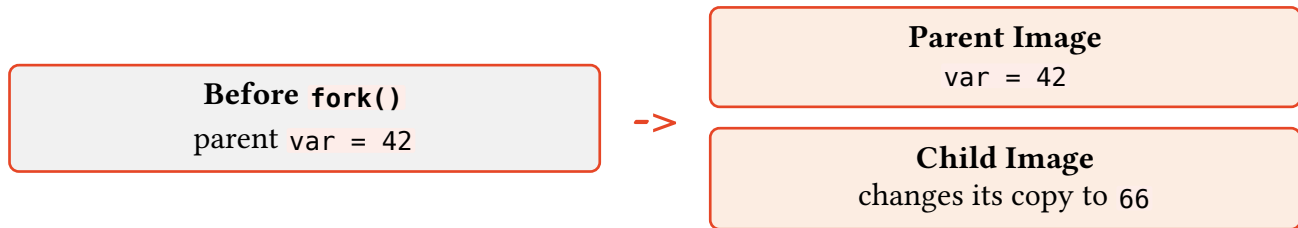
signal dispositions

open-file and pipe references

Note: Ordinary locals, globals, and heap objects belong to one process image.

Reveal: `fork()` Copies Values, Not One Shared Object

8 / 77



Note: The parent's ordinary `partial` remains `0`. The child needs an IPC path to return `66`.

One `fork()` Call Creates Two Continuing Paths

9 / 77

<code>fork()</code> Returns	Meaning
-1	Creation failed; no child
0	This is the child path
> 0	This is the parent; value is child PID

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork");
} else if (pid == 0) {
    /* child path */
} else {
    /* parent path */
}
```

Note: Source-code order does not guarantee parent/child scheduling order.

How many processes do you have if you run the following code block?

```
fork();  
fork();  
fork();
```

You don't need to remember them word by word, just make sure you know the difference between them, and could use anyone of them in the correct senecio.

```
int execlp(const char *file, char *const argv[]);
int execl(const char *path, char *const argv[]);
int execlp(const char *file, const char *arg,... (char *) NULL);
int execl(const char *path, const char *arg,... (char *) NULL);
int execlpe(const char *file, char *const argv[],char *const envp[]);
int execlpe(const char *path, const char *arg, ..., (char *) NULL, char * const
envp[] );
```

Example Usage of `execl()`

12 / 77

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    puts("about to launch sort");

    if (execl("/usr/bin/sort", "sort", "forkdemo.c", NULL) == -1) {
        perror("exec failed");
        return 1;
    }

    puts("finished sort");
    return 0;
}
```

What Executes After a Successful `exec()`?

13 / 77

```
if (pid == 0) {  
    execl("/bin/echo", "echo",  
          "ready", (char *)NULL);  
    perror("execl");  
    _exit(127);  
}
```

**On success, does
`perror()` run?**

Same PID, new program image.

No. Explain the reason.

wait() and waitpid()

14 / 77

`wait()` and `waitpid()` do not wait for “another process” in general. They wait for a child of the calling process.

```
pid_t pid = fork();

if (pid == 0) {
    printf("HC: hello from child\n");
} else if (pid > 0) {
    printf("HP: hello from parent\n");
    wait(NULL); // Equiv to waitpid(pid, NULL, 0);
    printf("CT: child has terminated\n");
}
```

`exec()` Replaces; `waitpid()` Observes

15 / 77

After `fork()`, the two lanes continue independently.

Child Lane

close/redirect FDs -> `exec*()` -> new program

Parent Lane

`wait()` / `waitpid()` -> inspect status

Successful `exec*()` Replaces the process image and does not return. Open FDs normally survive unless marked close-on-exec.

Child selection `wait()` selects an eligible child; `waitpid(pid, ...)` selects the requested child.

W8 Tutorial B Exercise: Bash Simulator

16 / 77

Import header files and start the main loop.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define BUFFER_SIZE 1024
#define MAX_ARGS 64

int main(void) {
    char line[BUFFER_SIZE];
    while (1) {
        ...
    }
}
```

W8 Tutorial B Exercise: Bash Simulator (ii)

17 / 77

Handle the arguments and convert the input into tokens.

```
...
printf("mini$ ");
fflush(stdout);

if (fgets(line, sizeof line, stdin) == NULL) {
    printf("\n");
    break;
}

line[strcspn(line, "\n")] = '\0';

if (line[0] == '\0') continue;

char *argv[MAX_ARGS];
int argc = 0;
```

W8 Tutorial B Exercise: Bash Simulator (iii)

18 / 77

```
char *token = strtok(line, " \t");

while (token != NULL && argc < MAX_ARGS - 1) {
    argv[argc++] = token;
    token = strtok(NULL, " \t");
}

argv[argc] = NULL;
if (argc == 0) continue;
if (strcmp(argv[0], "exit") == 0) break;
...
```

W8 Tutorial B Exercise: Bash Simulator (iv)

19 / 77

Fork the process, replace the program in the child process, and pass arguments.

```
...
pid_t pid = fork();
if (pid < 0) {
    perror("fork failed");
    continue;
}
if (pid == 0) {
    execvp(argv[0], argv);
    perror("execvp failed");
    _exit(127);
} else {
    if (waitpid(pid, NULL, 0) == -1) {
        perror("waitpid failed");
    }
}
```

W8 Tutorial B Exercise: Bash Simulator (v)

20 / 77

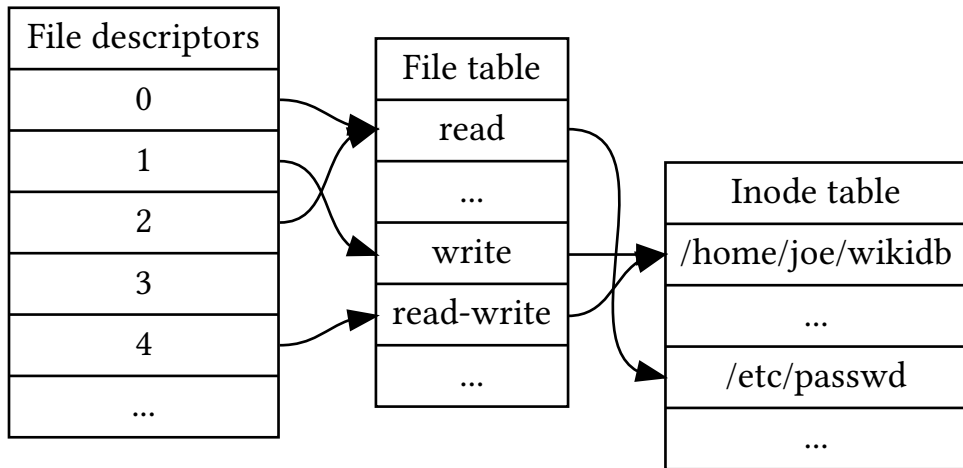
```
}  
    return 0;  
}
```

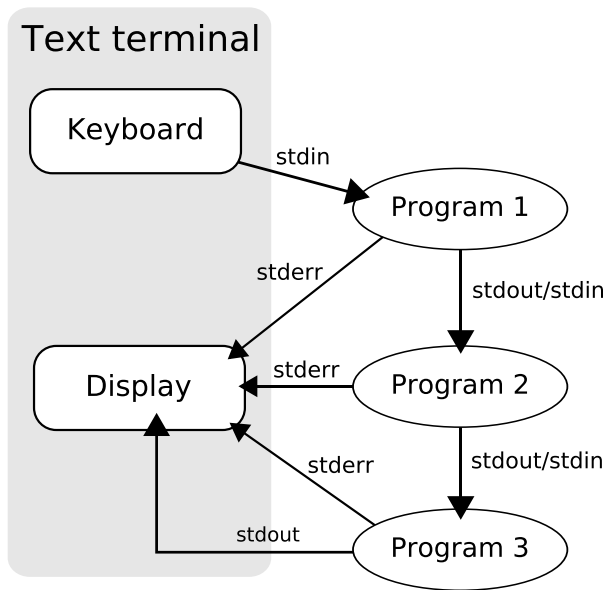
For detailed explanations, please visit [the corresponding chapter in my tutorial notes](#).

2 IPC and Process Workers

A File Descriptor Is a Handle, Not the Data

22 / 77





FD	Process-Local Meaning
0	standard input
1	standard output
2	standard error
3	pipe read end
4	pipe write end

->

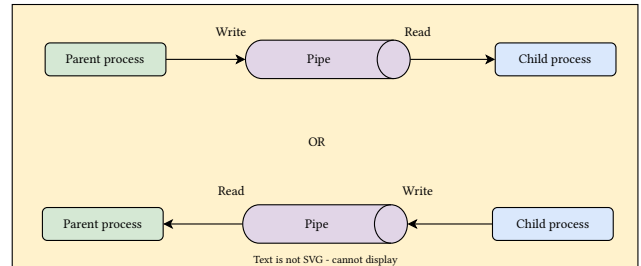
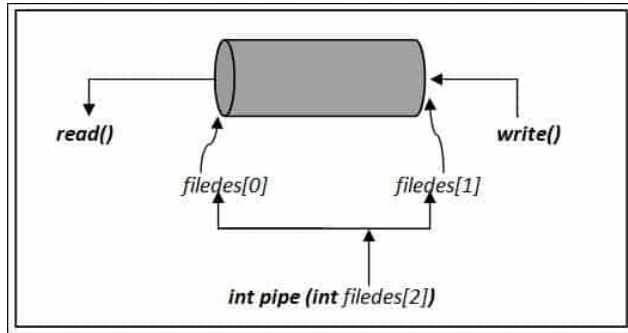
terminal / file open description

kernel pipe object
read endpoint + write endpoint

Note: Example FDs only: `pipe()` returns the lowest available numbers. Each integer indexes a process table entry that refers to a kernel object.

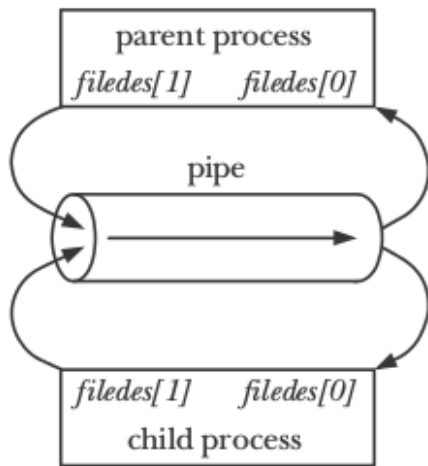
pipe()

25 / 77

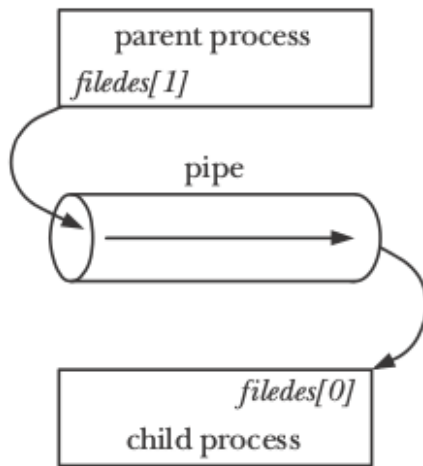


Forked Processes Reference the Same Pipe

26 / 77



a) After `fork()`

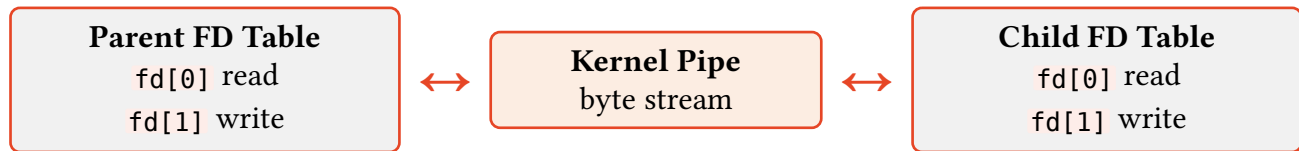


b) After closing unused descriptors

Figure 44-3: Setting up a pipe to transfer data from a parent to a child

Forked Processes Reference the Same Pipe (ii)

27 / 77



Parent keeps `fd[0]` for reading; closes its inherited `fd[1]`.

Child keeps `fd[1]` for writing; closes its inherited `fd[0]`.

Data direction: child `fd[1]` -> kernel pipe -> parent `fd[0]`.

Reveal: Pipe EOF and Complete Transfer

28 / 77

Child closed its writer. Parent still holds a writer. EOF?

A pipe is a byte stream: One `write()` does not promise one complete matching `read()`.

Retry partial transfers and `EINTR`; treat early EOF as incomplete data.

No, one write-end reference remains open.

Note: Close every unused endpoint, then use complete-transfer helpers for fixed records.

Example Usage of Processes and Pipelines

29 / 77

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    int pipefd[2];
    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }
    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        return 1;
    }
    ...
}
```

Example Usage of Processes and Pipelines (ii)

30 / 77

```
...
// Child: redirect stdout to the pipe.
if (pid == 0) {
    // Child: redirect stdout to the pipe.
    close(pipefd[0]); // child does not read
    if (dup2(pipefd[1], STDOUT_FILENO) == -1) {
        perror("dup2");
        return 1;
    }
    // fd 1 now refers to the pipe, so this is no longer needed
    close(pipefd[1]);
    printf("hello from child\n");
    fflush(stdout);
    return 0;
}
...
```

Example Usage of Processes and Pipelines (iii)

31 / 77

```
...
// Parent: read from the pipe.
close(pipefd[1]); // parent does not write
char buffer[256];
ssize_t nread = read(pipefd[0], buffer, sizeof(buffer) - 1);
if (nread == -1) {
    perror("read");
    close(pipefd[0]);
    waitpid(pid, NULL, 0);
    return 1;
}
buffer[nread] = '\0';
printf("parent received: %s", buffer);
close(pipefd[0]);
waitpid(pid, NULL, 0);
return 0;
}
```

Design the Process Solution Before System Calls

32 / 77

Worker 0

[0, 3)

partial 6

Worker 1

[3, 6)

partial 15

Worker 2

[6, 9)

partial 24

Worker 3

[9, 11)

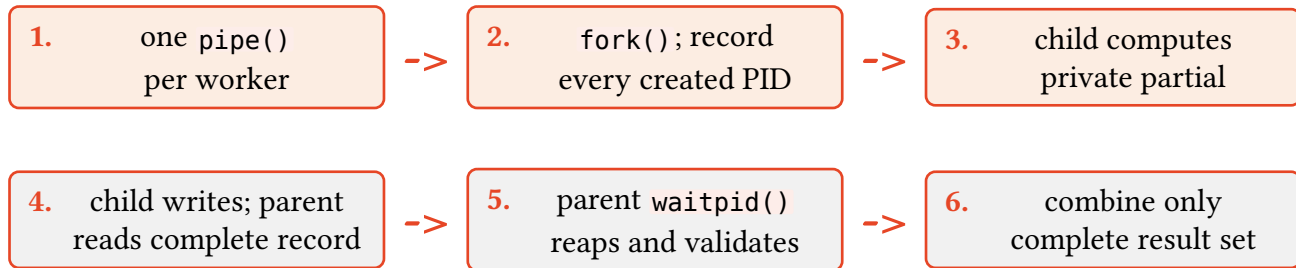
partial 21

How will each partial reach the parent, and how will the parent know it is final?

Half-open ranges: adjacent boundaries, no gaps, no overlap, final end 11.

The Process Result Path Has Six Explicit Stages

33 / 77



Note: Transport and completion are separate arrows; a strong answer names both.

Child Path: One Range, One Result Channel

34 / 77

```
if (pid == 0) {
    close(result_fd[i][0]);
    close_unrelated_fds(result_fd, i);

    int64_t partial =
        sum_range(begin, end);
    int out = result_fd[i][1];
    if (write_full(out, &partial,
                  sizeof partial) < 0)
        _exit(1);
    close(out);
    _exit(0);
}
```

Child Obligations

- keep one range;
- keep one write endpoint;
- compute privately;
- transfer exactly one record;
- close and terminate.

The Parent Collects, Reaps, Validates, Then Combines

35 / 77

Collect Every Created Result

```
for(size_t i=0; i < created; i++) {  
    data_ok[i] =  
    read_result(fd[i],  
    &partial[i]);  
    close(fd[i]);  
}
```

Reap Every Created Child

```
for(size_t i = 0; i < created; i++) {  
    int status;  
    pid_t got =  
    waitpid_nointr(child[i], &status);  
    child_ok[i] = child_exited_ok(got,  
    child[i], status);  
}
```

Note: Reap every created child. Publish only if all intended workers, records, and statuses are valid; otherwise recover or reject.

Can the Parent Wait Before Draining a Large Pipe?

36 / 77

Parent

`waitpid(child, ...)`
waits for termination

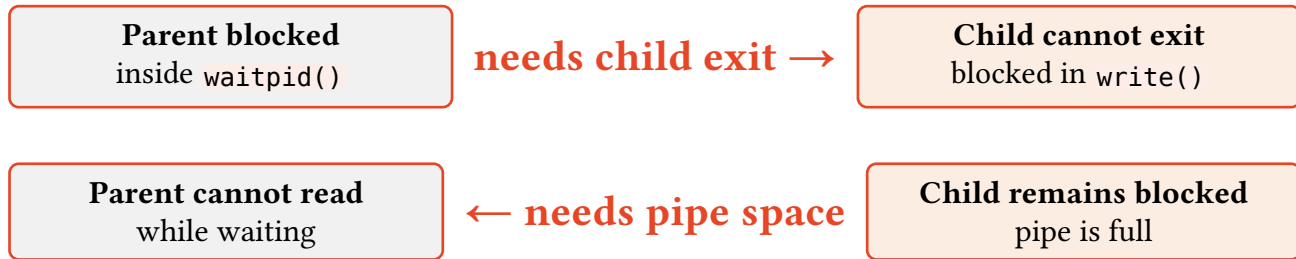
Child

`write(large_result)`
may wait for pipe space

If the pipe becomes full, who can perform the next enabling action?

Wait-Before-Read Can Create a Progress Cycle

37 / 77



Note: Drain output before a wait that could strand the producer. With several streams, observe readiness and then read.

Choose IPC by What Must Cross the Boundary

38 / 77

Need	Mechanism	Defining Semantic
Small event notification	<code>sigaction()</code> + signal	Notify; keep handler minimal
One-way byte stream	<code>pipe()</code>	Kernel transfers bytes
Several active FDs	<code>select()</code> / <code>poll()</code> / <code>epoll</code>	Report readiness; then call I/O
Large addressable region	<code>mmap(..., MAP_SHARED, ...)</code>	Shared visibility; add synchronization
Child state and reaping	<code>waitpid()</code>	Lifecycle, not arbitrary data

Choose by the information crossing the boundary, not by a familiar API name.

Signal

`signal()` is legacy shorthand. Prefer `sigaction()` with explicit mask and flags.

Handler sets a volatile `sig_atomic_t` flag; ordinary control flow does work.

```
#include <signal.h>

static volatile sig_atomic_t
got_sigint = 0;

static void handle_sigint(int signum)
{
    (void)signum;
    got_sigint = 1;
}
```

The handler receives the signal number. In this example, the handler only sets a flag. That is a good habit because signal handlers should stay small.

```
struct sigaction sa;

sa.sa_handler = handle_sigint;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;

if (sigaction(SIGINT, &sa, NULL) == -1) {
    perror("sigaction");
}
```

- when `SIGINT` arrives, call `handle_sigint`
- while the handler is running, block only the default extra signals from `sa_mask`
- use `SA_RESTART` behaviour where possible

Readiness

`select`: rebuilt FD sets.

`poll`: `pollfd` array.

`epoll`: persistent Linux interest set.

Shared Mapping

Create anonymous `MAP_SHARED` before `fork()`.

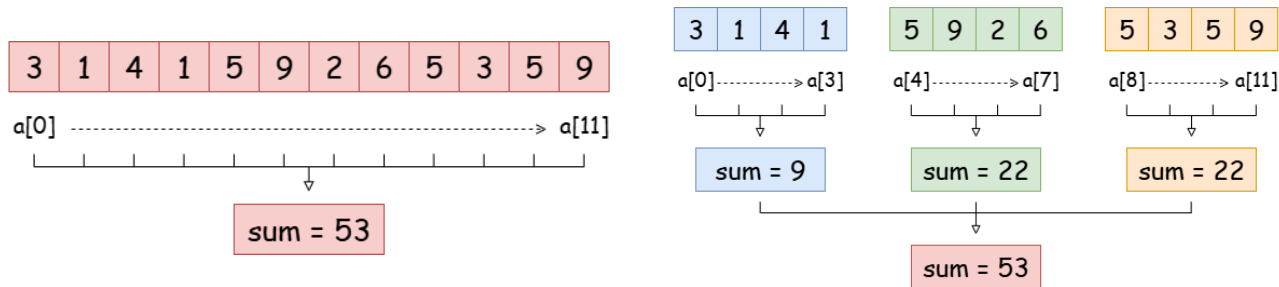
`malloc()` / `MAP_PRIVATE` stay private. Add synchronization.

Every result path still needs framing, validation, and cleanup.

3 Threads and Worker-Combiner

Use Threads to Run Small Tasks in Parallel

43 / 77



Prototype: Create Threads in C

44 / 77

```
int pthread_create(pthread_t *thread,  
                  const pthread_attr_t *attr,  
                  void *(*start_routine)(void *),  
                  void *arg);
```

// thread where the new thread ID is stored
// attr thread attributes; usually NULL for defaults
// start_routine function the new thread will run
// arg argument passed to that function

Threads Share the Address Space, Not Execution State

45 / 77

One process boundary

shared code, globals, heap, and file descriptors

T0
stack

T1
stack

T2
stack

Shared

globals

heap

FD table

Separate per thread

stack

registers

program counter

Note: Shared visibility removes IPC copies; it does not remove race conditions.

Worker-Combiner Makes Ownership Visible

46 / 77



Coverage invariant Every input index is in exactly one half-open range.

Ownership invariant Each result slot has one worker writer; main reads after join.

Note: Prefer private partials plus a combiner over locking every update.

Why is Passing `&i` From the Loop Unsafe?

47 / 77

```
for (size_t i = 0; i < N; i++) {  
    pthread_create(&thread[i], NULL, worker, &i); // wrong  
}
```

How many distinct addresses did the workers receive?

loop: `i = 0 -> 1 -> 2 -> 3 -> 4`
worker may first run here

Stable Task Records Give Every Worker One Result Slot

48 / 77

```
struct task {  
    const int *values;  
    size_t begin, end;  
    int64_t result;  
};
```

```
static void *worker(void *arg) {  
    struct task *t = arg;  
    int64_t local = 0;  
    for (size_t i = t->begin; i < t->end; i++)  
        local += t->values[i];  
    t->result = local;    // one writer  
    return NULL;  
}
```

Note: Stable lifetime + distinct record address + one writer + join-before-read.

Which Loop Shape Preserves Parallel Overlap?

49 / 77

A: immediate join

```
for each i:  
  create(i)  
  join(i)
```

B: two phases

```
for each i: create(i)  
for each i: join(i)
```

Which shape lets workers overlap in time?

Create the Set, Join the Created Set, Then Combine

50 / 77

Execution lanes

main: create 0 | create 1 | create 2

T0: ===== partial 9

T1: ===== partial 22

T2: ===== partial 22

main: join created set -> validate coverage ->
combine 53

Job	Processes	Threads
Result	pipe/shared mapping	shared task slot
Complete	waitpid()	pthread_join()
Cleanup	close + reap	join created set

Simple Example of Threads in C

51 / 77

For more instructions, you could visit my notes on https://github.com/ninn-kou/COMP2017-Tutorials/blob/main/Week%2010/Week_10_Tutorial_B.md.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void *worker(void *arg) {
    (void)arg;
    printf("hello from worker thread\n");
    return NULL;
}
```

Simple Example of Threads in C (ii)

52 / 77

```
int main(void) {
    pthread_t thread;
    int err = pthread_create(&thread, NULL, worker, NULL);
    if (err != 0) {
        fprintf(stderr, "pthread_create: %s\n", strerror(err));
        return EXIT_FAILURE;
    }
    printf("hello from main thread\n");
    err = pthread_join(thread, NULL);
    if (err != 0) {
        fprintf(stderr, "pthread_join: %s\n", strerror(err));
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

4 Race Conditions and Mutexes

Can Two Increments Succeed but Increase Only Once?

54 / 77

Initial shared value
total = 7

**Two threads
execute
total++. Must
the result be 9?**

Step	Thread A	Thread B
1	read ?	
2		read ?
3	compute ?	
4		compute ?
5	write ?	
6		write ?

A Legal Interleaving can Lose One Update

55 / 77

Step	Thread A	Thread B	Shared total
1	read 7		7
2		read 7	7
3	compute 8		7
4		compute 8	7
5	write 8		8
6		write 8	8

Race condition Outcome depends on uncontrolled timing/interleaving. The table illustrates one lost update.

Data race Threads access one C object without synchronization and at least one writes: undefined behavior.

```
#include <pthread.h>

static int x = 0;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increase(void *arg) {
    (void)arg;

    pthread_mutex_lock(&lock);
    x = x + 1;
    pthread_mutex_unlock(&lock);

    return NULL;
}
```

Find the Invariant, Then Protect Its Critical Section

57 / 77

Before choosing a lock

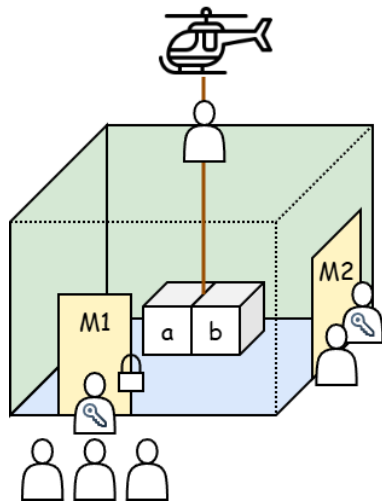
1. What state is shared?
2. What invariant must remain true?
3. Which reads and writes participate?
4. Can ownership remove the sharing?

```
int64_t local = compute(task);  
  
pthread_mutex_lock(&total_lock);  
shared_total += local;    // locked  
pthread_mutex_unlock(&total_lock);
```

Note: Critical section = the complete reads, decisions, and writes that preserve a shared invariant. Every conflicting access follows one mutex protocol; `join()` is only completion.

One Diagram to Remember

58 / 77



The thing being locked is the storage cage, not the shared resource.

Multiple resources (a and b) could be protected by one lock.

One person can be treated as one thread. Only one thread can access the shared resource at a time: the one that holds the key.

One lock is like one door. It only restricts the threads that use that access door (M1). If other threads use another door (M2), they can interfere at any time.

Only the thread that locked the door can unlock it (1 key for 1 door).

If a thread does not use a lock at all (by helicopter), it always has access to the shared resource.

5 Deadlocks

How A Simple Deadlock Happens?

60 / 77

```
pthread_mutex_t M1;  
pthread_mutex_t M2;
```

Thread 1 locks them in this order:

```
pthread_mutex_lock(&M1);  
pthread_mutex_lock(&M2);  
  
/* do work */  
  
pthread_mutex_unlock(&M2);  
pthread_mutex_unlock(&M1);
```

Thread 2 locks them in the opposite order:

```
pthread_mutex_lock(&M2);  
pthread_mutex_lock(&M1);  
  
/* do work */  
  
pthread_mutex_unlock(&M1);  
pthread_mutex_unlock(&M2);
```

How A Simple Deadlock Happens? (ii)

61 / 77

The dangerous part is the inconsistent lock acquisition order:

T1: M1 then M2

T2: M2 then M1

T1 locks M1

T2 locks M2

T1 tries to lock M2, but T2 has it

T2 tries to lock M1, but T1 has it

T1 holds M1 and waits for M2

T2 holds M2 and waits for M1

Neither thread can continue. T1 cannot unlock M1 because it is blocked waiting for M2. T2 cannot unlock M2 because it is blocked waiting for M1.

Can These Two Lock Sequences Trap Each Other?

62 / 77

Pseudocode using `lock` / `unlock` shorthand for pthread mutex operations.

```
void catalogue_asset(void) {  
    lock(&metadata);  
    lock(&storage);  
    update_catalogue();  
    unlock(&storage);  
    unlock(&metadata);  
}
```

```
void relocate_asset(void) {  
    lock(&storage);  
    lock(&metadata);  
    move_asset();  
    unlock(&metadata);  
    unlock(&storage);  
}
```

Give one exact interleaving in which neither thread reaches unlock.

Four Legal Steps are Enough to Deadlock

63 / 77

Step	Action	Tcat holds / waits	Trel holds / waits
1	Tcat locks metadata	metadata / none	none / none
2	Trel locks storage	metadata / none	storage / none
3	Tcat requests storage	metadata / storage	storage / none
4	Trel requests metadata	metadata / storage	storage / metadata

Can either owner reach the release that the other needs?

No: this is a closed set of blocked owners.

The Wait-For Cycle Makes the Dependency Explicit

64 / 77

Tcat
holds metadata

↓ waits for storage

Trel
holds storage

↑ waits for metadata

Closed directed cycle

Coffman condition	Evidence here
Mutual exclusion	each mutex has one owner
Hold and wait	each thread holds one and requests one
No preemption	a mutex is released by its owner
Circular wait	Tcat waits for Trel; Trel waits for Tcat

The Four Conditions For Deadlock

65 / 77

Break any one of them could prevent the deadlock.

1. Mutual exclusion

At least one resource can only be held by one thread at a time.
A mutex is exactly this kind of resource.

2. Hold and wait

A thread is holding one resource while waiting for another.

3. No preemption

A lock cannot be forcibly taken from a thread.
The thread that owns the lock must voluntarily unlock it.

4. Circular wait

A cycle of waiting exists.
For example, T1 waits for T2, and T2 waits for T1.

A Global Lock Order Makes Circular Wait Impossible

66 / 77

One published order metadata_lock
(rank 1)

↓ **acquire a higher rank**

storage_lock (rank 2)

Every path strictly increases rank.

Why the repair works Assume a wait cycle exists.

Every edge goes from a held lower rank to a requested higher rank.

Ranks must strictly increase around the cycle, yet return to the start: impossible.

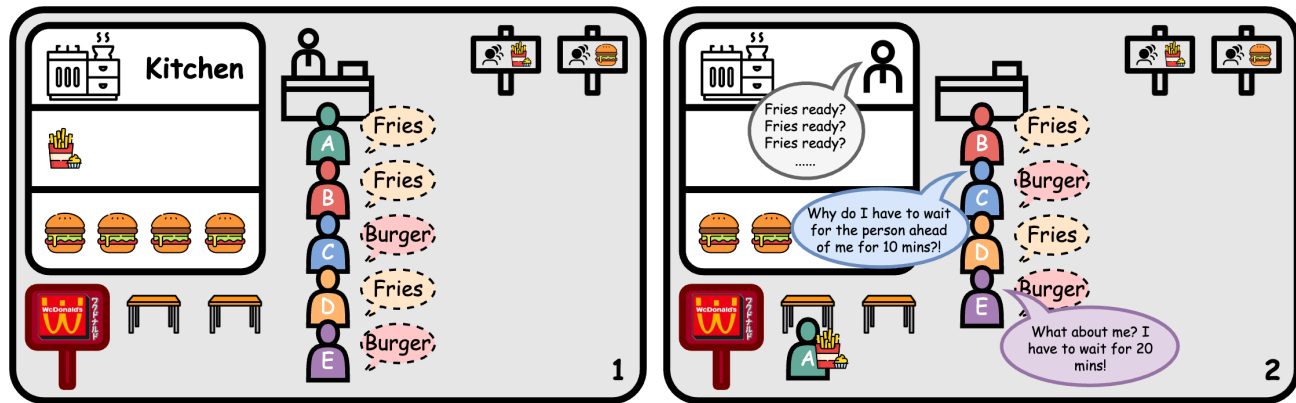
Race: unsafe access **Contention:** owner can progress **Deadlock:** closed blocked dependency

6 Condition Variables and Semaphores

Semaphores: WcDonald's

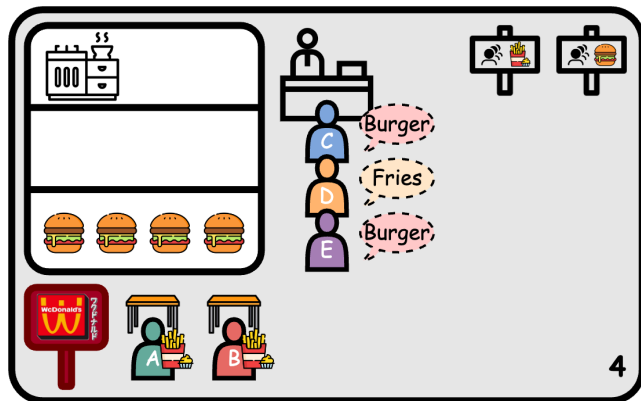
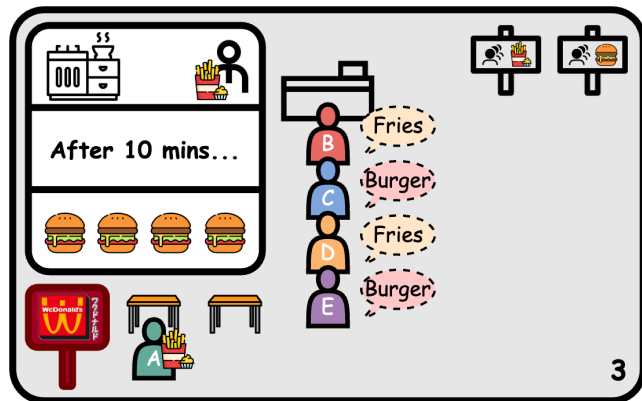
68 / 77

For detailed explanations, please refer to https://github.com/ninn-kou/COMP2017-Tutorials/blob/main/Week%2012/Week_12_Tutorial_A.md#a2-semaphores.



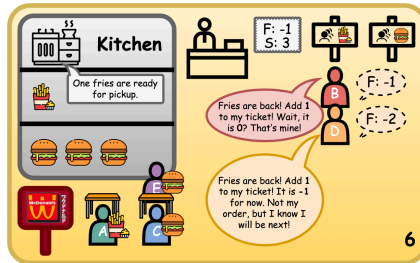
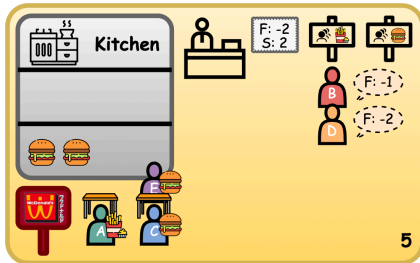
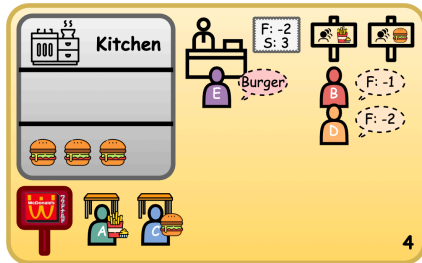
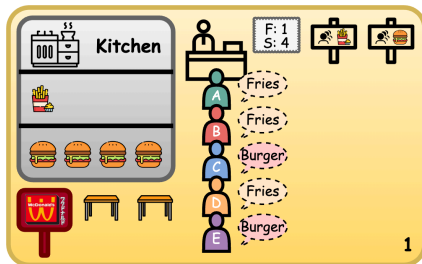
Semaphores: WcDonald's (ii)

69 / 77



Semaphores: McDonald's (iii)

70 / 77



A Condition Variable Waits for a Predicate

71 / 77

```
pthread_mutex_lock(&m);  
while (queue_empty())  
    pthread_cond_wait(&not_empty,  
                      &m);  
  
item = queue_remove();  
pthread_mutex_unlock(&m);
```

1. lock and test predicate
2. wait releases mutex atomically
3. producer locks `m`, changes state, signals, unlocks
4. waiter reacquires and retests

Semaphore: the count persists.

Condition variable: a notification is not stored state.

Choose a Primitive by the Relationship You Need

72 / 77

Question	Primitive	Use when...
Who owns one invariant?	mutex	one participant at a time enters the protocol
Many readers or one writer?	read-write lock	reads dominate and the invariant tolerates concurrent readers
How many units are available?	semaphore	permits, items, or slots must be counted
Is a predicate true yet?	condition variable + mutex	a thread should sleep until protected state may have changed

API memory jog `pthread_rwlock_rdlock`: concurrent readers; `wrlock`: exclusive writer.
`sem_wait`: consume or block at zero; `sem_post`: add a permit.

7 Recursion and Exam Transfer

Recursion Needs a Base Case and Strict Progress

74 / 77

`sum_to(3) = 3 + sum_to(2)`

`sum_to(2) = 2 + sum_to(1)`

`sum_to(1) = 1 + sum_to(0)`

`sum_to(0) = 0` base case

unwind: 0 -> 1 -> 3 -> 6

Four Questions

1. What is the base case?
2. What measure strictly decreases?
3. What subproblem is called?
4. How are returned values combined?

Parallel Recursion

Independent branches + cutoff + bounded workers + join before combine.

Recursions with Multi-Threads: W12 Tut B - Fibonacci Numbers - III

75 / 77

https://github.com/ninn-kou/COMP2017-Tutorials/blob/main/Week%2012/Week_12_Tutorial_B.md#b4-exercise-fibonacci-numbers---iii

Rebuild the Answer from the Obligation

76 / 77

Obligation	Likely Mechanism	One Sentence that Earns Clarity
Create a worker	<code>fork()</code> / <code>pthread_create()</code>	state whether memory is separate or shared; pass stable task state
Return a child result	<code>pipe()</code> / shared mapping	name the explicit process data path
Observe completion	<code>waitpid()</code> / <code>pthread_join()</code>	reap or join the exact successfully created set
Protect a live invariant	<code>mutex</code> / <code>rwlock</code> / <code>semaphore</code> / <code>condvar</code>	choose by the required synchronization relationship
Repair deadlock	global lock order	state the order and prove the wait cycle impossible

These slides may be **updated dynamically** over the coming days, so please always check this page for the latest version.

Additionally, please note that our workshop does not cover all of the semester's content. For more detailed explanations, you can visit <https://github.com/ninn-kou/COMP2017-Tutorials>.

Thank you for your participation today. Good luck with your P2P exam!